

CSSE3113 – Introduction to Software Engineering

Software Process Models

Software Development

Construction

Some of the major construction activities are listed below.

- Requirement Gathering
- Design Development
- Coding
- Testing

Management

Some of the major management activities are listed below.

- Project Planning and Management
- Configuration Management
- Software Quality Assurance
- Installation and Training

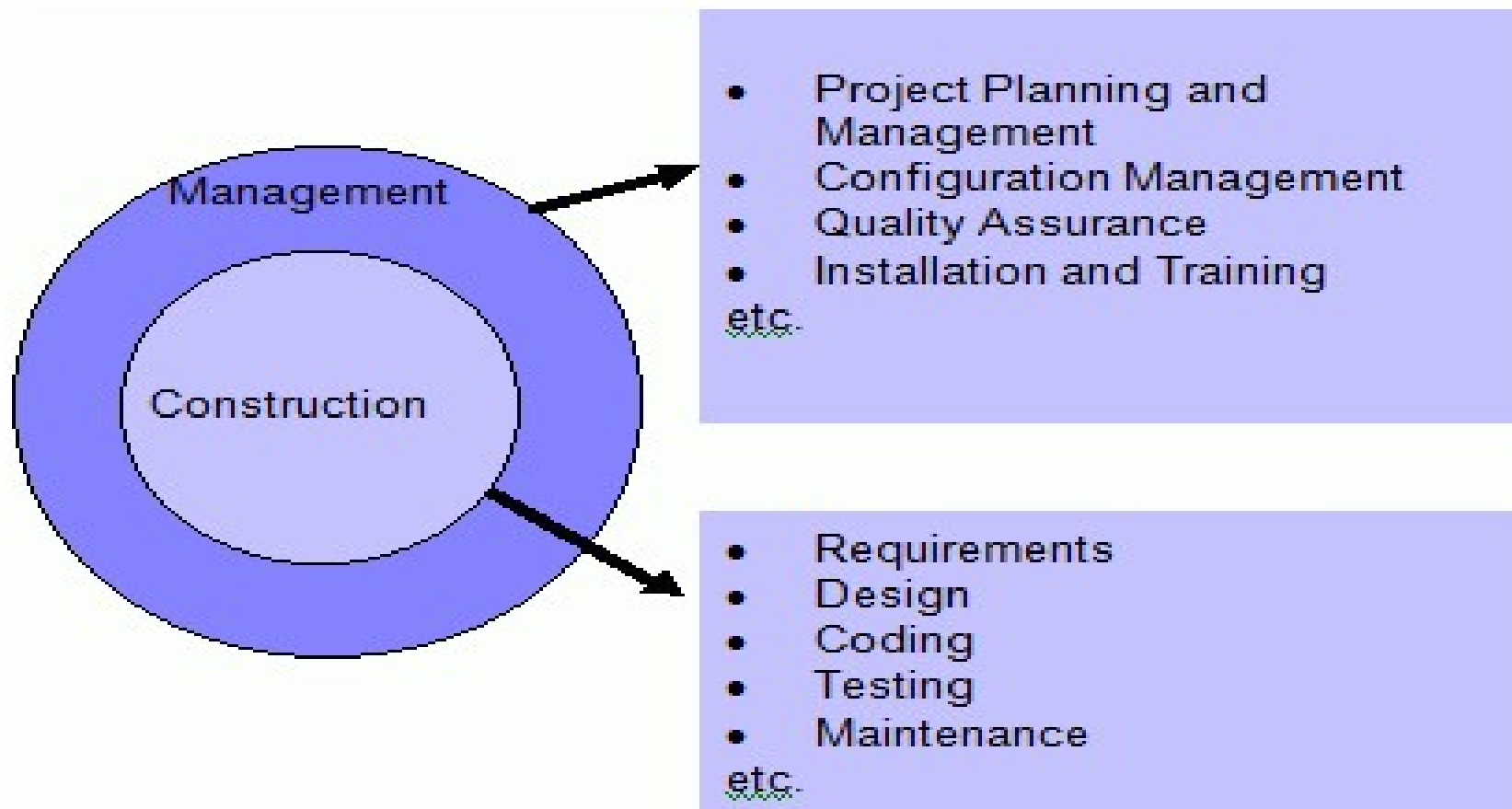


Figure 1: Development Activities

Ad hoc Software Development

- Developing software without planning for each phase, and without specifying tasks, deliverables, or time constraints.
- Relies entirely on the skills and experience of the individuals performing the work.
- The software process may change as work progresses.

Case Study : SaudiTech

What could go wrong in a software development project?

SaudiTech is a software company. It has a team of **25 programmers**. Ali has recently established a dental clinic in Riyadh and asked SaudiTech to develop a management system for his dental clinic.

Ali's request was to develop a system to electronically manage patient records and perform administrative functions similar to 'what is being used in other clinics' but within a budget of **500 Saudi riyals** and delivered within **3 months**.

SaudiTech adopts an ad hoc approach (**Developing software without planning for each phase, and without specifying tasks, deliverables, or time constraints**) to software development and has not worked on large scale projects before.

What problems do you anticipate in this project?

How can we overcome problems with ad hoc software development?

➤ **Problems:**

- Difficult to distinguish between tasks → important tasks may be ignored.
- Inconsistent schedules, budgets, functionality and product quality.
- Delayed problem discovery → more costly to fix.

Solution? Software Process Model

- Software Process Models provide guidelines to organize **how** software process activities should be performed and **in what order**.

The Software Process

- A **structured set of activities** required to develop a software system.
- Many different software processes but all involve:
 - **Specification** – defining what the system should do
 - **Design and implementation** – defining the organization of the system and implementing the system;
 - **Validation** – checking that it does what the customer wants;
 - **Evolution** – changing the system in response to changing customer needs.
- A **software process model** is an **abstract representation** of a process. It presents a description of a process from some particular perspective.

Software Process Descriptions

- When we describe and discuss processes, we usually talk about the **activities** in these processes such as specifying a data model, designing a user interface, etc. and the ordering of these activities.
- Process descriptions may also include:
 - **Products**, which are the **outcomes** of a process activity;
 - **Roles**, which reflect the responsibilities of the people involved in the process;
 - **Pre- and post-conditions**, which are statements that are true before and after a process activity has been enacted or a product produced.

Plan-driven and Agile Processes

- **Plan-driven processes** are processes where all of the process activities are planned in advance and progress is measured against this plan.
 - Plan drives everything!
- In Agile Processes planning is incremental and it is **easier to change** the process to reflect changing customer requirements.

Plan-driven and Agile Processes

- Activity

Research and list two advantages and two disadvantages of both Plan-Driven and Agile processes.

Plan-driven and Agile Processes

- In practice, most practical processes include elements of both plan-driven and agile.
- There are no right or wrong software processes.

Factor	Plan-Driven	Agile
Project Type	Fixed requirements	Evolving requirements
Flexibility	Low	High
Customer Involvement	Low (feedback at the end)	High (continuous feedback)
Budget & Timeline Predictability	High	Medium
Documentation	Extensive	Minimal
Risk of Late Changes	High	Low
Best for	Large-scale, compliance-heavy projects	Rapid, user-driven development

Software Process/SDLC Models

Software Process model:

- Abstract representation of a software process
- Each Process model represents a process from different perspective

- Following are the most important and popular SDLC models followed in the industry:
 - Waterfall model.
 - V Model
 - Evolutionary development.
 - Component-Based development model (CBSE).
 - Process iteration(incremental and spiral Models)

The other related methodologies are Agile and RAD Model – Rapid Application Development

Waterfall Model

- First published model
- Because of the cascade from one phase to another, this model is known as 'Waterfall Model'.
- It is also referred to as linear-sequential life cycle model.
- Each phase must be completed before the next phase can begin and there is no overlapping in the phases.

Waterfall Model

The sequential phases in Waterfall model are:

Requirement Gathering and analysis: All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.

- System's services
- Constraints
- Goals

System Design: The requirement specifications from first phase are studied in this phase and system design is prepared. System Design helps in specifying hardware and system requirements and also helps in defining overall system architecture.

Waterfall Model

Implementation and unit testing: With inputs from system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality which is referred to as Unit Testing.

Integration and Testing: All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.

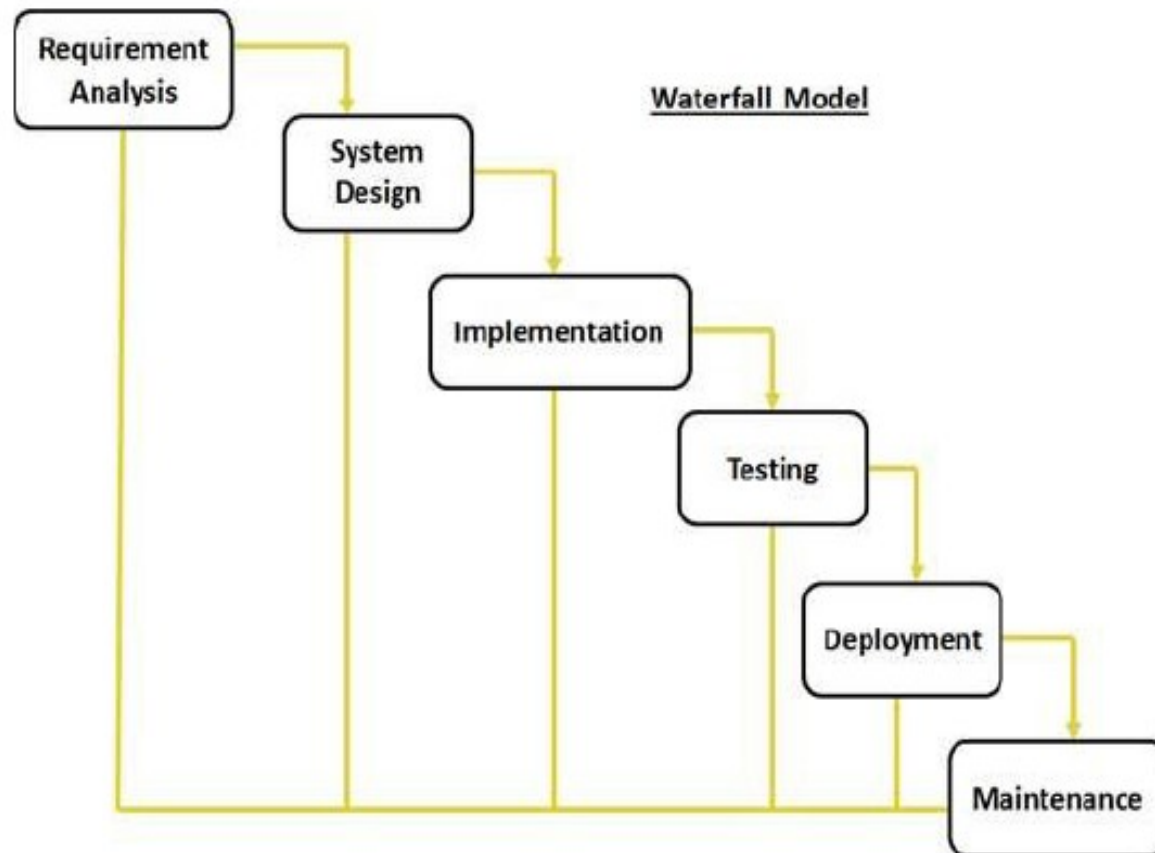
Waterfall Model

Deployment of system: Once the functional and nonfunctional testing is done, the product is deployed in the customer environment or released into the market.

Maintenance: There are some issues which come up in the client environment. To fix those issues patches are released. Also, to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

Waterfall Model

Following is a diagrammatic representation of different phases of waterfall model.



Waterfall Model

Applicability:

- Requirements are very well understood, clear and fixed
- Product definition is stable
- Technology is understood

Waterfall Model

Advantages:

- Documentation - at each phase
- Phases are processed and completed one at a time.
- Easy to arrange tasks.

Problems:

- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.

V Model

- A variation of the waterfall model
- Development phases are presented in a V-shaped graph.
- It is also known as **Verification and Validation model**.
- For every **single** phase in the development cycle there is a directly associated testing phase.

Verification and Validation

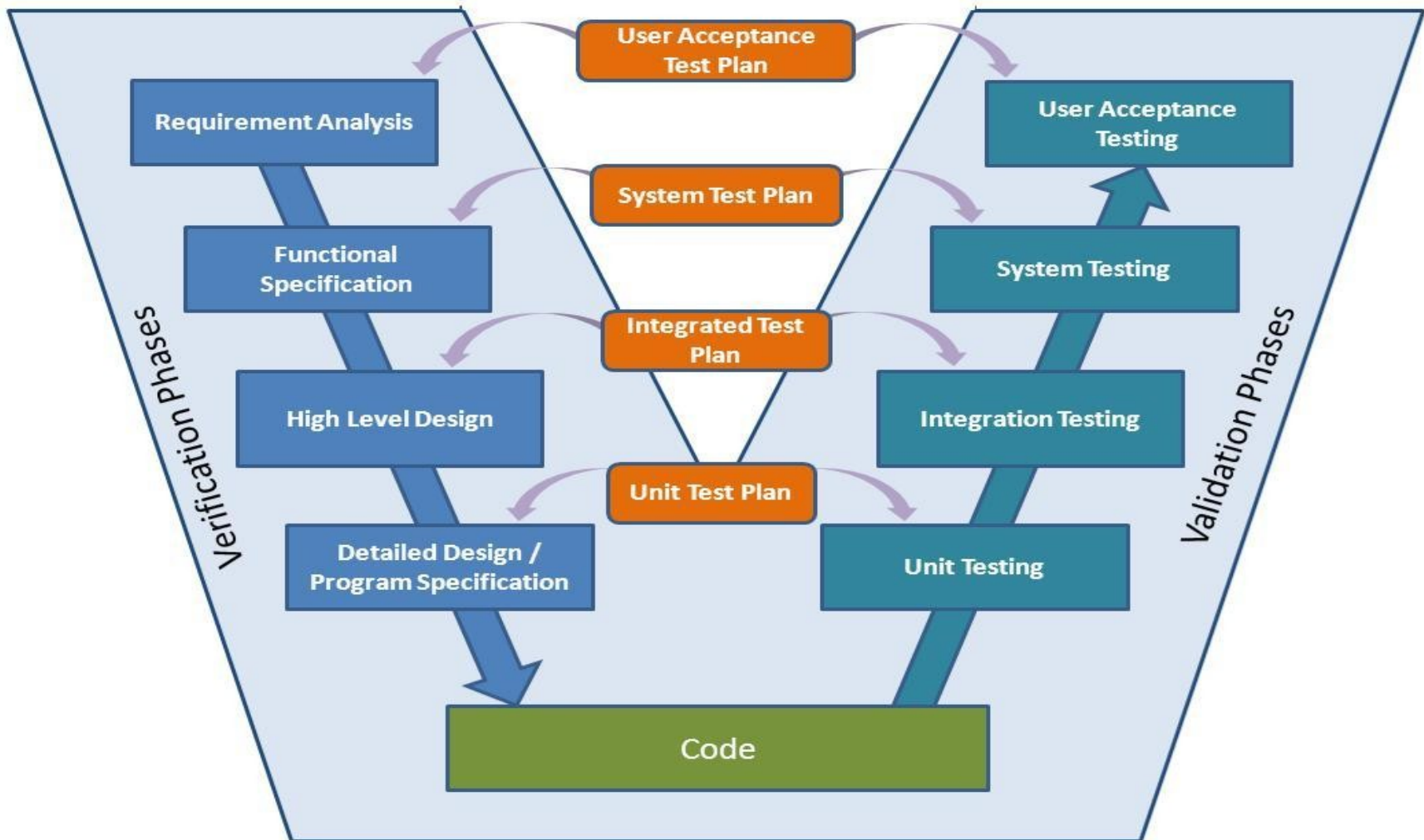
- ***Verification***

- Does the product meet system specifications?
- Have you built the product right?

- ***Validation***

- Does the product meet user expectations?
- Have you built the right product?

V Model



V Model Verification Phases

Following are the Verification phases in V-Model

Business Requirement Analysis:

- This phase involves detailed communication with the customer to understand his expectations and exact requirement.
- The acceptance **test** design planning is done at this stage as business requirements can be used as an input for acceptance testing.

System Design:

- System design would comprise of understanding and detailing the complete hardware and communication setup for the product under development.
- **System test plan** is developed based on the system design.

V Model Verification Phases

Architectural Design:

- System design is broken down further into modules taking up different functionality.
- The data transfer and communication between the internal modules and with the outside world (other systems) is clearly understood and defined in this stage.
- With this information, integration tests can be designed and documented during this stage.

Module Design:

- In this phase the detailed **internal design** for all the system modules is specified
- It is important that the design is compatible with the other modules in the system architecture and the other external systems.
- Unit tests can be designed at this stage

V Model

Coding Phase

- The actual coding of the system modules designed in the design phase is taken up in the Coding phase.
- The best suitable **programming** language is decided based on the system and architectural requirements.
- The coding is performed based on the coding guidelines and standards.
- The code goes through numerous code reviews and is optimized for best performance before the final build is checked into the repository.

V Model Validation Phases

Following are the Validation phases in V-Model:

Unit Testing:

- Unit tests designed in the module design phase
- Unit testing is the testing at code level and helps to eliminate bugs at an early stage

Integration Testing:

- Integration testing is associated with the architectural design phase.
- Integration tests are performed to test the coexistence and communication of the internal modules within the system.

V Model Validation Phases

System Testing:

- System tests check the entire system functionality and the communication of the system under development with external systems.
- Most of the software and hardware compatibility issues can be uncovered during system test execution.

Acceptance Testing:

- Acceptance testing is associated with the business requirement analysis phase and involves testing the product in user environment.
- It also discovers the non functional issues such as load and performance defects in the actual user environment.

V Model

Advantages:

- Testing activities like planning, test designing, happens well before coding.
- Higher chances of success over the waterfall model.
- Works well for projects where requirements are easily understood.

Problems:

- Difficult to handle changes in requirements throughout the life cycle

V Model

Applicability:

- The V-shaped model should be used for projects where requirements are clearly defined and fixed.
- The V-Shaped model should be chosen when technical resources are available with needed technical expertise.
- High confidence of customer is required for choosing the V-Shaped model approach.

Evolutionary Models

Evolutionary Development:

- Includes the activities of specification, development and validation
- Initial system is developed from abstract specification
- Then refined with customer input to produce a system that satisfies the customer's needs.

Two types of evolutionary development

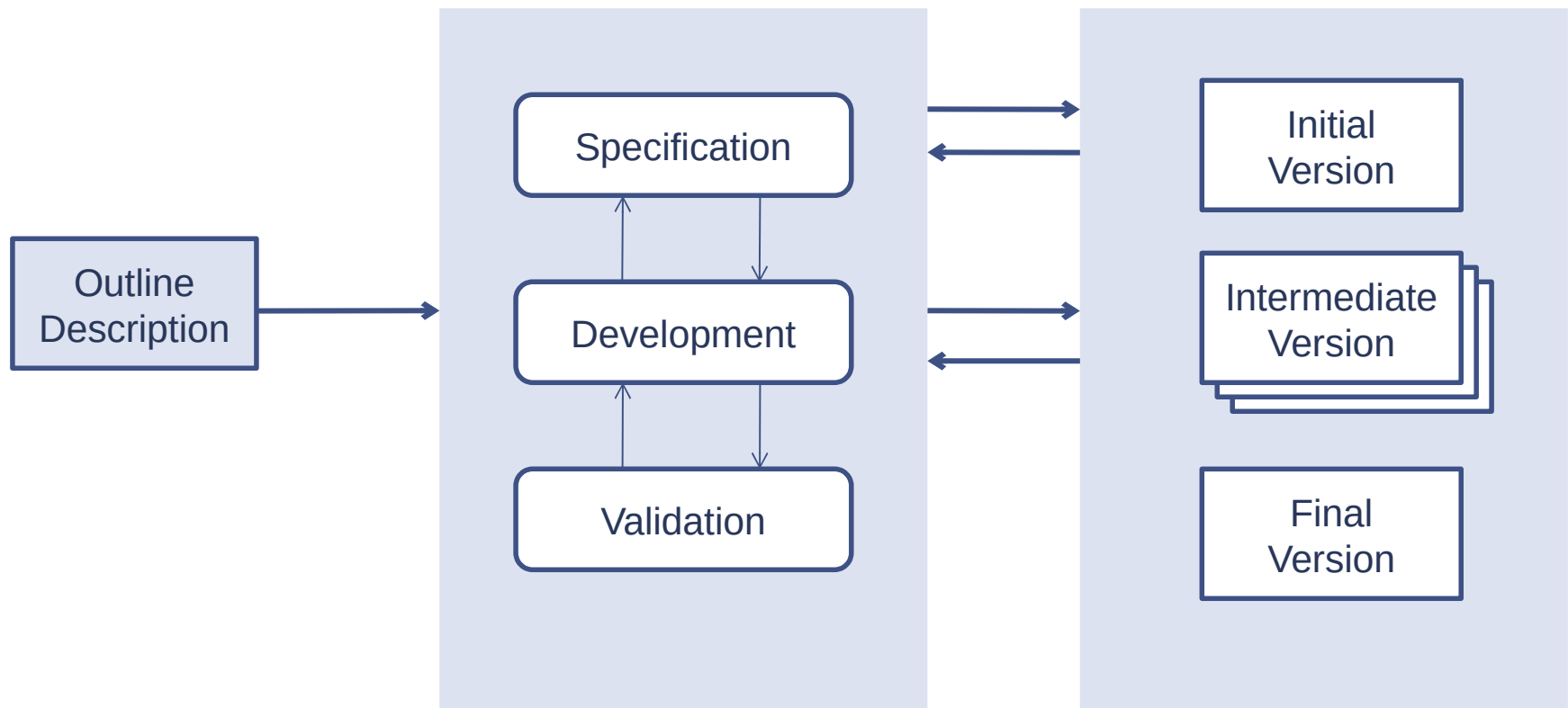
1. Exploratory
2. Prototyping

Evolutionary Models

Exploratory Model

- Objective of the process is to work with customers to explore their requirements and deliver a final system.
- The development starts with the parts of the system that are understood.
- The system evolves by adding new features proposed by customer.
- This model works best in situations where few, or none, of the system or product requirements are known in detail.

Exploratory Models



Exploratory Model

- **Problems:**
 - Lack of process visibility;
 - Systems are often poorly structured
- **Applicability:**
 - For small or medium-size interactive systems
 - For parts of large systems (e.g. the user interface)

Evolutionary Models

Prototyping Model

- When a customer defines a set of general objectives for a software but does not identify **detailed I/O** or **processing requirements**.
- A prototype is built to understand the requirements.
- By using this prototype, the client can get an “actual feel” of the system
- The interactions with prototype can enable the client to better understand the requirements of the desired system
- Prototyping is an attractive idea for complicated and large systems.

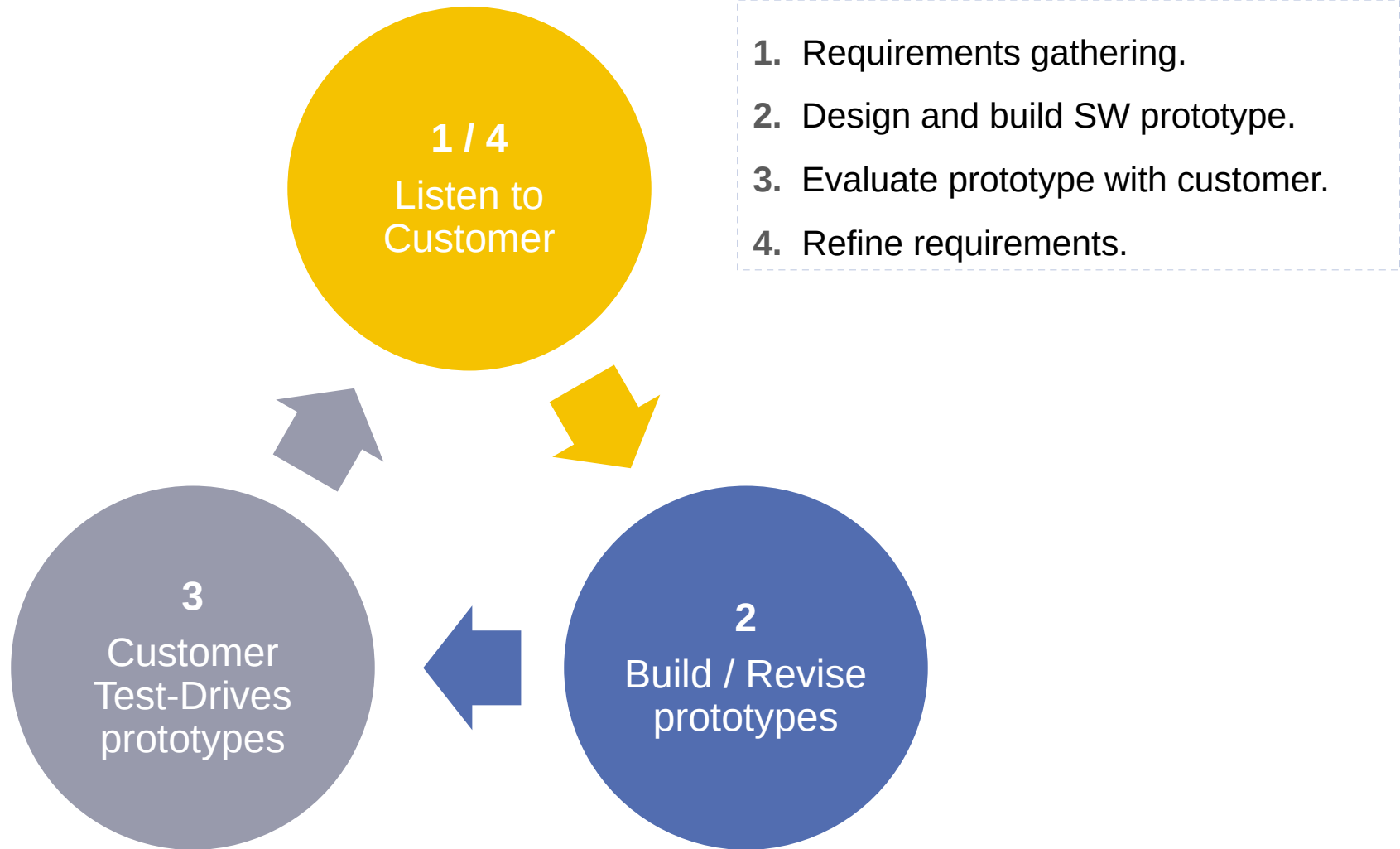
Evolutionary Models

Prototyping Model

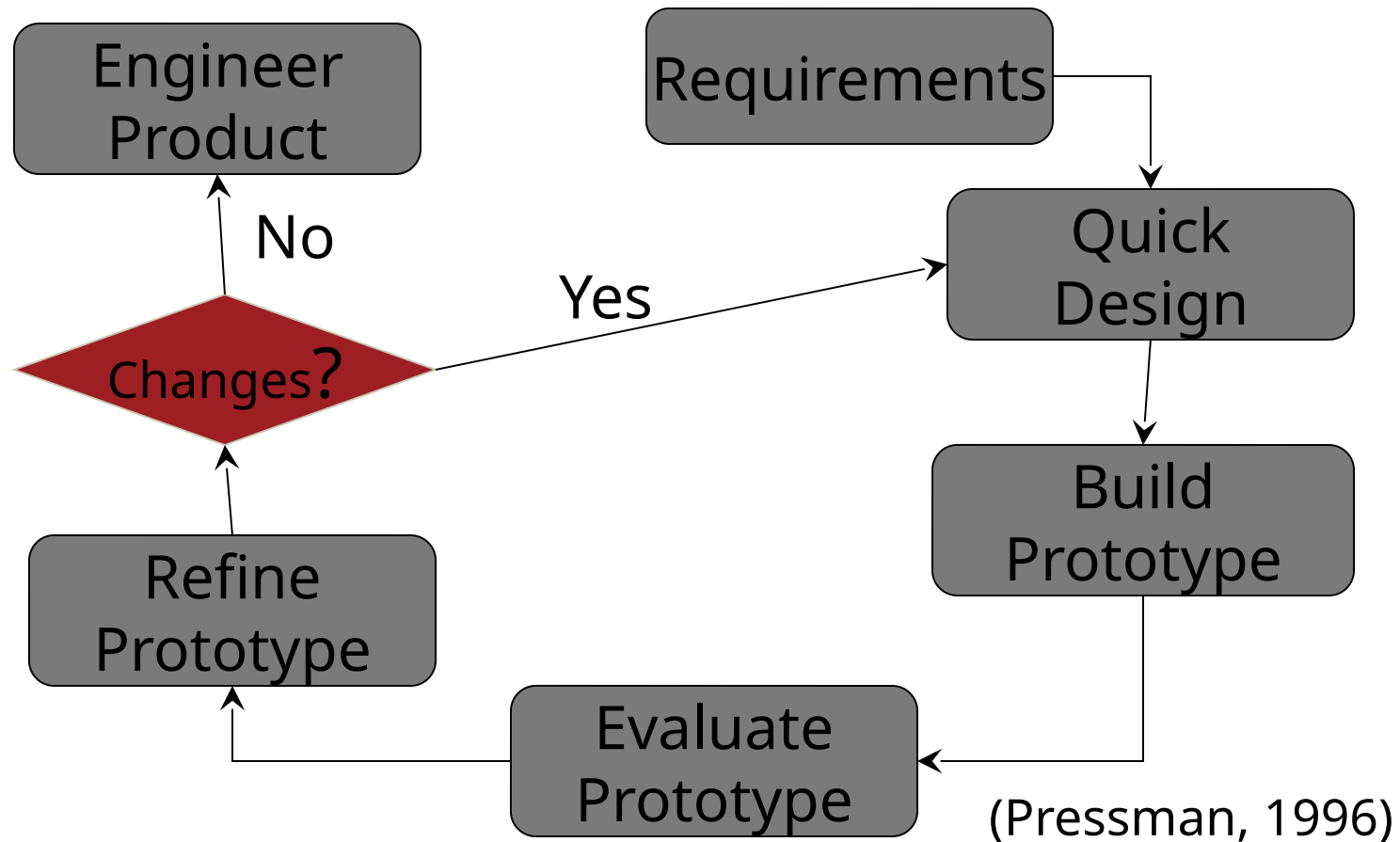
Consists of 4 iterating phases:

- Requirements gathering.
- Design and build prototype.
- Evaluate prototype with customer.
- Refine requirements.

Prototype Model



Prototyping



Prototype Model

Advantages:

- Users are actively involved in the development
- Users get a better understanding of the system being developed.
- Errors can be detected much earlier.
- Quicker user feedback is available leading to better solutions.

Disadvantages:

- Practically, this methodology may increase the complexity of the system as scope of the system may expand beyond original plans.

Prototype Model

Applicability:

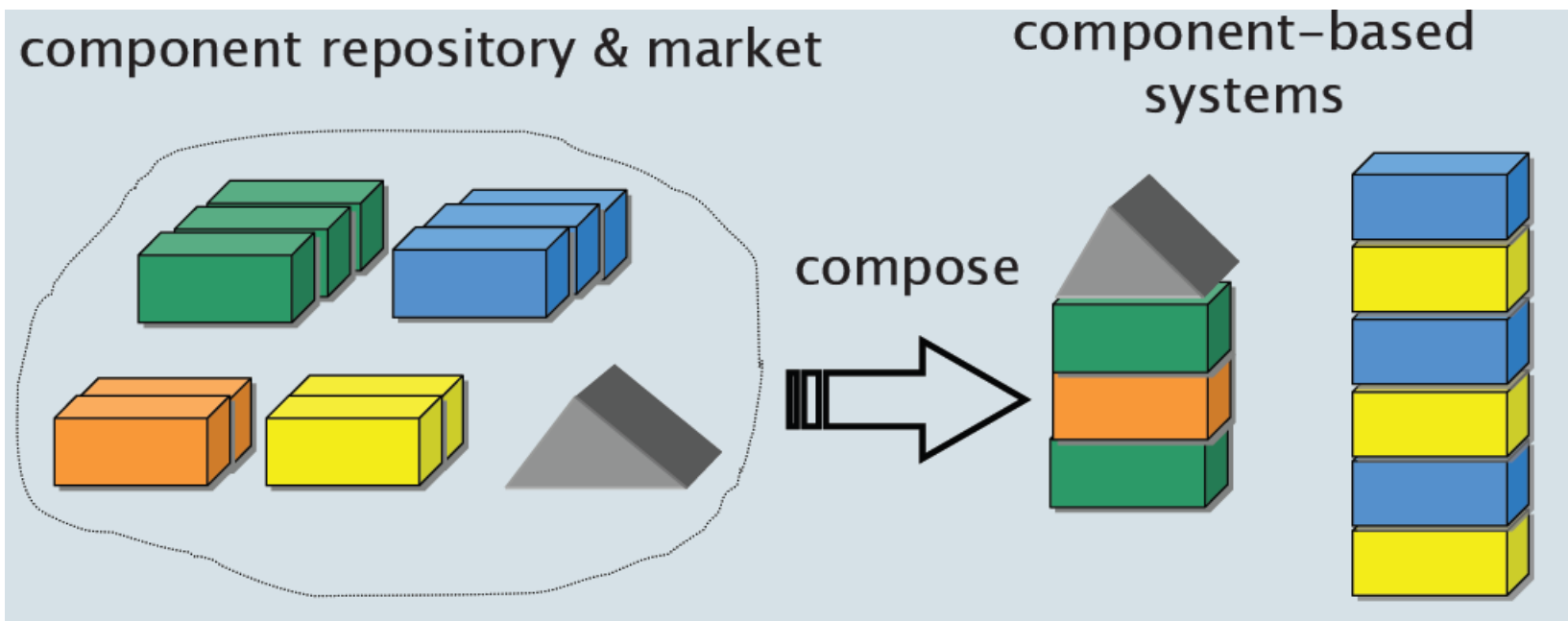
- Prototype model should be used when the desired system needs to have a lot of interaction with the end users.
- Typically, online systems, web interfaces have a very high amount of interaction with end users, are best suited for Prototype model.

Component-based Software Engineering

- Based on systematic reuse where systems are integrated from existing components.
- People working on the project → Know of design or code (similar to that required) → Modify them as needed → incorporate them into their system
- Process stages
 - Component analysis
 - Requirements modification
 - System design with reuse
 - Development and integration
- This approach is becoming increasingly used

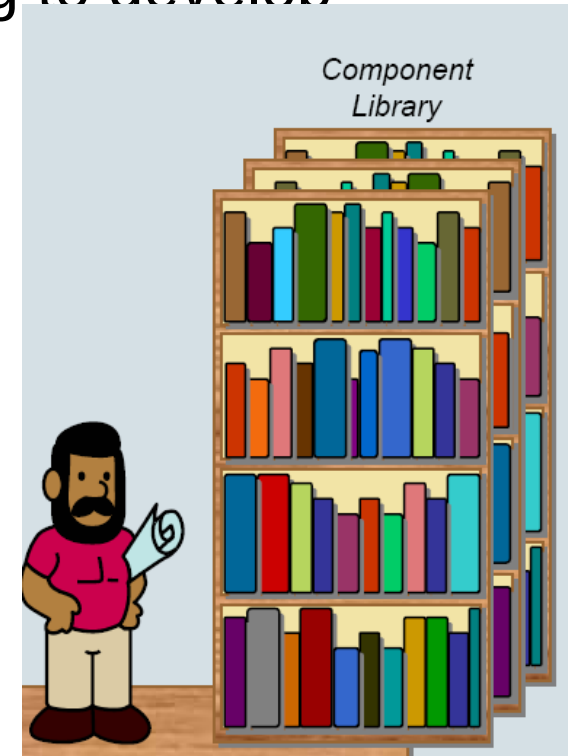
The CBD/CBSE is the Solution...

- Systems should be assembled from existing components

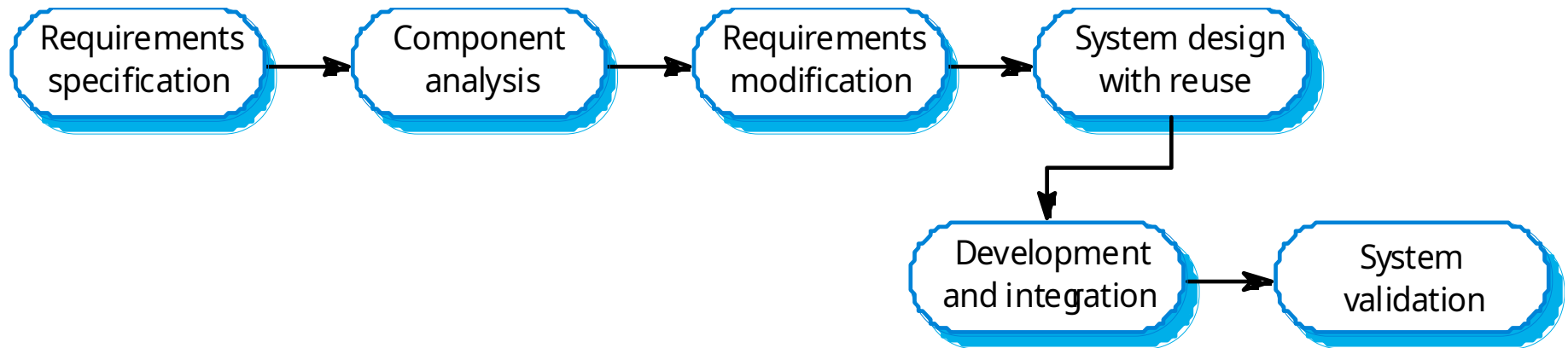


Why Components!!!

- Following other engineering disciplines (civil and electrical), software engineering is looking to develop
 - A catalogue of software building blocks
 - Connection Standards



Component-based Software Engineering



Component-based Software Engineering

Component analysis

- Given the requirement specification
- Search is made for components
- Usually there is no exact match.

Requirement Modification

- Requirements are analyzed—using information of discovered components
- If modifications are impossible— component analysis activity may be re-entered to search for alternate solution

System Design with Reuse

- Frame work of system is designed or an existing frame work is reused.

Development and integration Software

- Existing software /modified and newly developed components are integrated.

Component-based Software Engineering

Advantages

- Reduce the amount of software to be developed
- Reducing cost and risk
- Faster delivery of software

Disadvantages

- Requirement changes—may lead to a system that does not meet the real needs of users
- Control over the system evolution is lost

Process Iteration

Two (related) process models

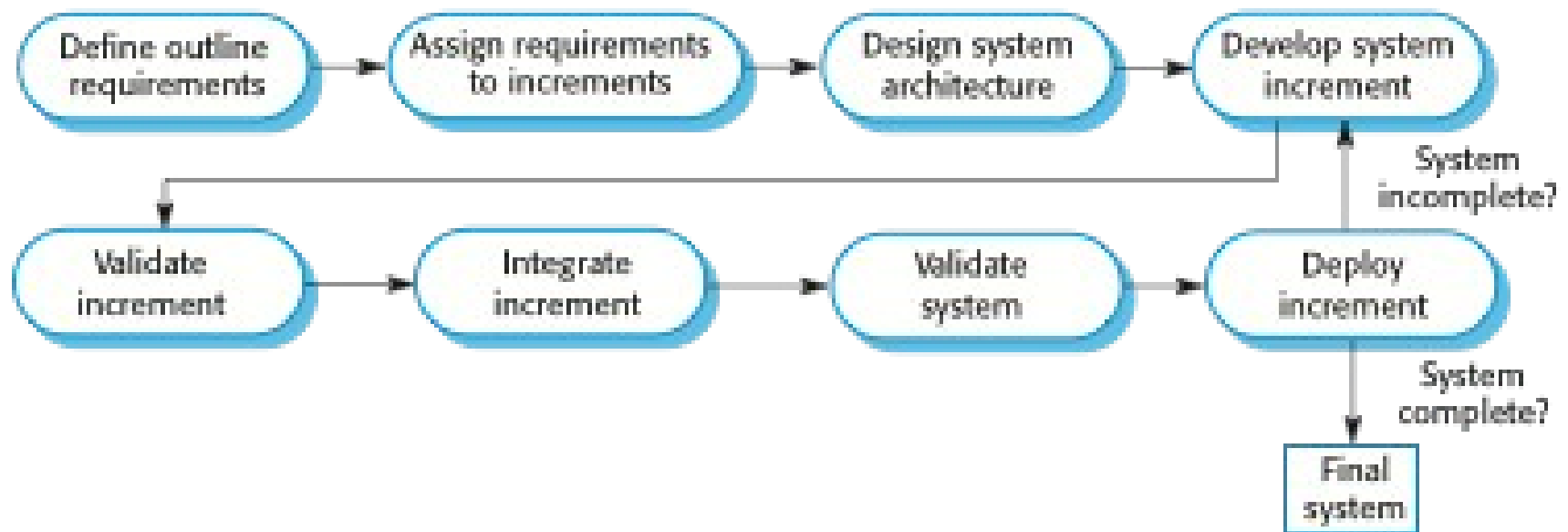
- Incremental delivery
- Spiral development

Incremental Approach

“The software specification, design and implementation are broken down into a series of increments that are each developed in turn”

- System development is decomposed into increments and each delivers a proportion of the system.
- Increments are developed based on their requirement priorities.

Incremental Development



Incremental Delivery(Steps)

1. Customer identify—services provided by the system
2. Then identify which of the services are most important and which are least important
3. No of delivery increments are then identified with subset of the system functionality
4. Highest priority services delivered first
5. Requirements for the first increment are defined in detail
6. First Increment is developed and delivered ..customer can put into service.(Benefit for customer?)
7. During developmentfurther requirements analysis for later increments can take place

Incremental Model

Advantages:

- Customer value can be delivered with each increment so system functionality is available earlier.
- Early increments act as a prototype to help elicit requirements for later increments.
- Lower risk of overall project failure.
- The highest priority system services tend to receive the most testing.

Disadvantages:

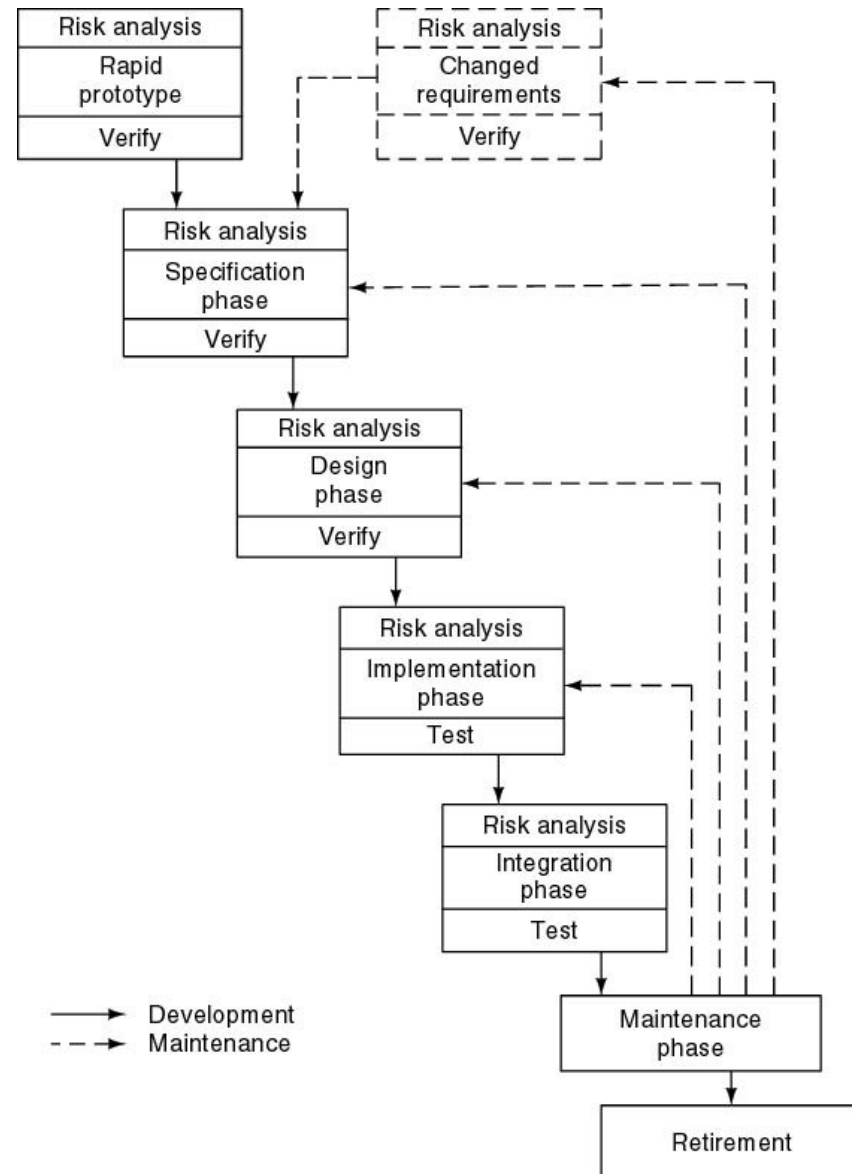
- Can be difficult to map the customer's requirements onto increments of the right size.
- Hard to identify common functions.

Spiral Model

- The software process is represented as spiral, rather than a sequence of activities with some backtracking from one activity to another.
- Each loop in spiral represents a phase of the software process
- Innermost loop might be concerned with system feasibility
- Next loop with requirements definition...next with system designed and so on.

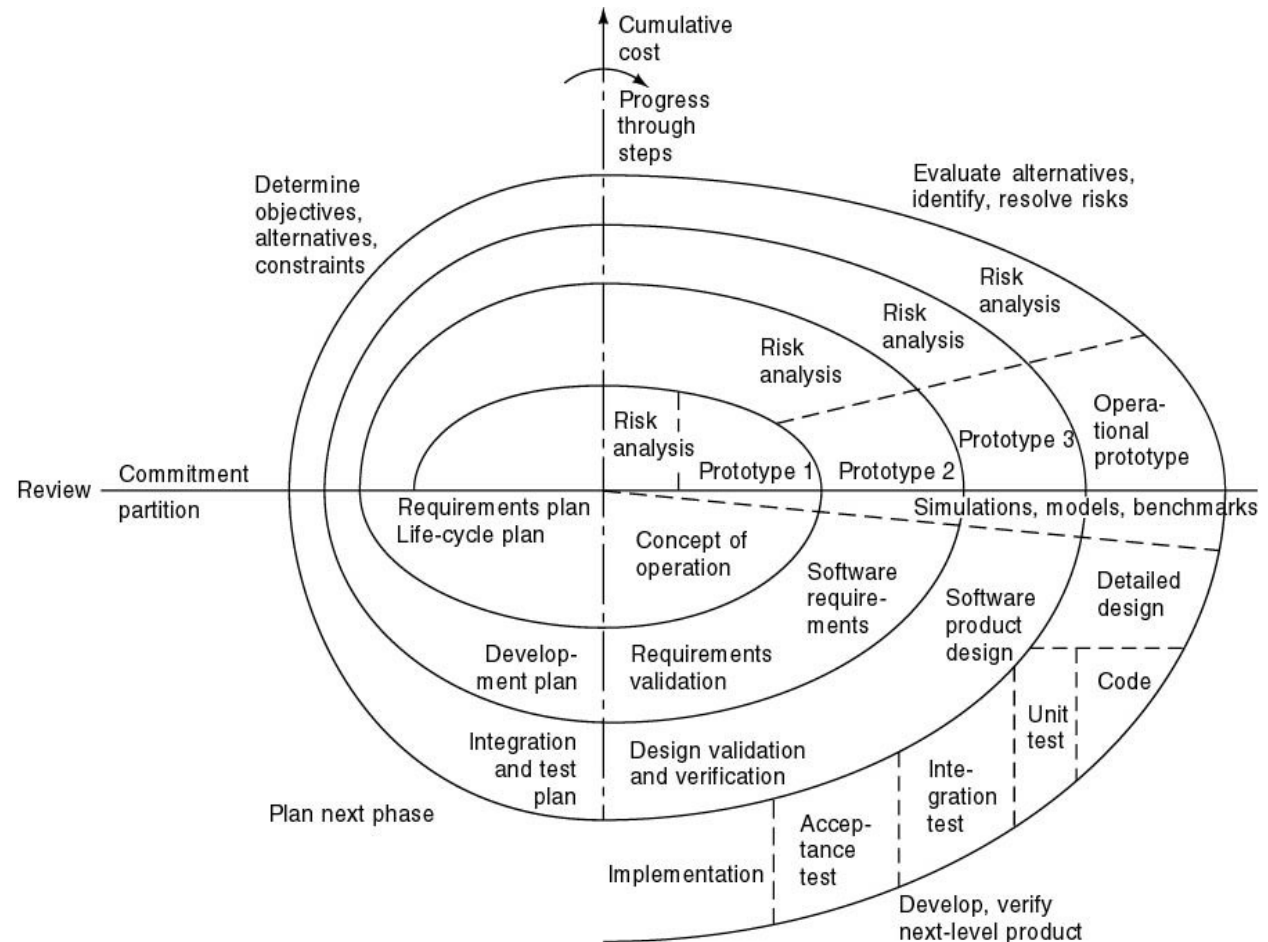
Spiral Model

- Simplified form
 - Waterfall model plus risk analysis
- Precede each phase by
 - Alternatives (build, reuse, buy, sub-contract)
 - Risk analysis (lack of experience, new technology, tight schedules)
- Follow each phase by
 - Evaluation
 - Planning of next phase

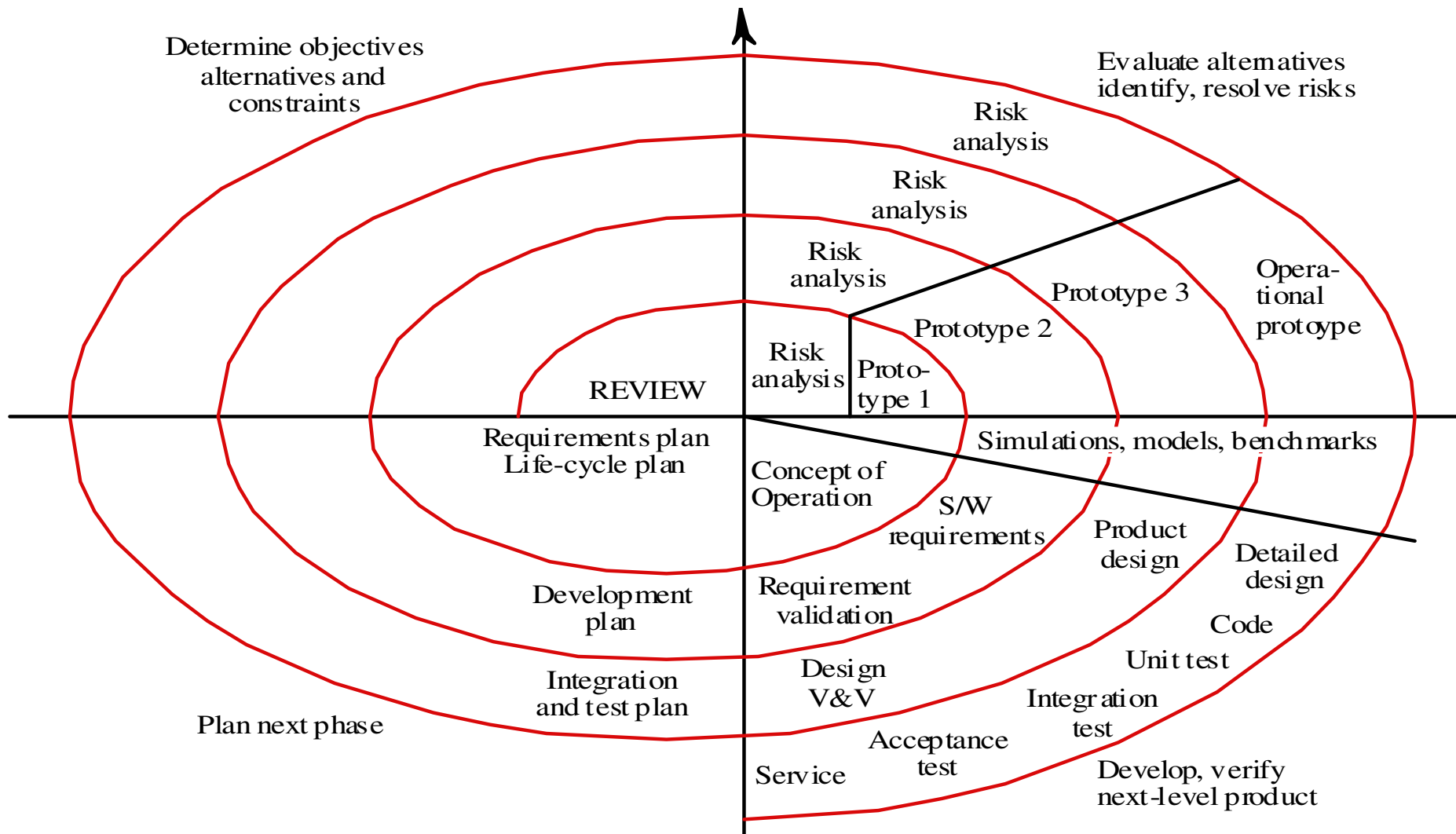


Full Spiral Model

- If risks cannot be resolved, project is immediately terminated
- Typical activities:
 - Create a design
 - Review design
 - Develop code
 - Inspect code
 - Test product



Spiral Model of the Software Process



Spiral Model Sectors

Each loop in spiral is split into four sectors

Objective setting

- Specific objectives for that phase of the project are identified
- Identify Constraints → on process & product.
- Detail management plan is drawn up
- Identify Project risks
- Alternative strategies on these risks ...may be planned

Spiral Model Sectors

Risk assessment and reduction

- For every identified risk → Detailed analysis is carried out
- Steps are taken to reduce the risk

E.g.

Risk :

Requirements are inappropriate

Solution / Strategy

A prototype system may be developed

* Risks are

- poorly understood requirements
- poorly understood architecture
- performance problems
- important missing features

Spiral Model Sectors

Development and validation

- A development model for the system is chosen which can be any of the generic models
- For example, if user interface risks are dominant, an appropriate model may be “Evolutionary Prototyping”.
- Selection of model will depend on your risk analysis.

Planning

- The project is reviewed
- Decision made whether to continue with a further loop of the spiral
- If decided to continue, the next phase of the spiral is planned

Spiral Model(Example)

- The evolution of Microsoft Operating System, Compilers and other operating systems.